

## CSCE 645 GEOMETRIC MODELING · MODULE 13 · LECTURE NOTES

# Deformation and Shape Editing

*Moving a handle and having the rest follow sensibly.*

## MODULE OUTCOMES

- Explain differential coordinates and why they encode detail.
- Implement Laplacian surface editing.
- Explain the rotation problem and how ARAP solves it.
- Explain cage-based and skeleton-based deformation.
- Choose a deformation method for a given task.

## 1 Why naive editing fails

The obvious approach to interactive editing is to displace a handle vertex and fall the displacement off smoothly over a neighbourhood. The result is a smooth bump, and every surface feature inside the affected region is flattened by it.

### DETAIL IS NOT STORED IN ABSOLUTE POSITIONS

Wrinkles, pores, creases, and tooling marks are **local** phenomena: each vertex sits a particular way relative to its immediate neighbours. That relationship is the detail.

Smoothly interpolating absolute positions preserves the large-scale shape and destroys the local relationships, which is exactly backwards.

The fix is to make the *differential* quantity the thing you preserve, and let absolute positions go wherever satisfying the constraints requires.

$$\delta_i = (\Delta \mathbf{x})_i = \mathbf{x}_i - \sum_j w_{ij} \mathbf{x}_j$$

This is the Laplacian of position from Module 08, now read differently: not as an operator to apply but as a **representation of shape**.  $\delta$  is large where the surface deviates from its local average — that is, where there is detail — and zero on a flat region. And by Module 07,  $\delta = 2Hn$ , so it encodes mean curvature directly.

## 2 Laplacian surface editing

Find new positions  $\mathbf{x}'$  that reproduce the original differential coordinates as closely as possible, while satisfying the handle constraints.

$$\text{minimise } \|\Delta \mathbf{x}' - \delta\|^2 \quad \text{subject to} \quad \mathbf{x}'_{\text{handles}} = \mathbf{p}$$

```
// Stacked least-squares system, with soft constraints:
//
//      [  L  ]      [ delta ]
//      [ w * C ] x' = [ w * p ]
//
// Normal equations:
//      (L^T L + w^2 C^T C) x' = L^T delta + w^2 C^T p
//
// Symmetric positive definite -> sparse Cholesky.
```

#### FACTOR ONCE, DRAG INTERACTIVELY

The system matrix  $L^T L + w^2 C^T C$  depends only on the mesh connectivity, the weights, and *which* vertices are constrained — not on *where* those vertices are being moved to.

So the expensive Cholesky factorisation is computed once when the user selects a handle, and each frame of dragging requires only a back substitution against a new right-hand side.

This is the Module 08 pattern delivering its clearest payoff: a solve that would take seconds becomes a few milliseconds, and detail-preserving editing becomes genuinely interactive. It is why this formulation found its way into modelling packages.

## 2.1 The rotation problem

#### $\delta$ IS A WORLD-SPACE VECTOR, AND THAT IS THE FLAW

The differential coordinate is a vector expressed in global coordinates. If part of the surface rotates, the detail vectors ought to rotate with it — a wrinkle on a bending arm should bend too.

The linear system has no way to express that. It preserves  $\delta$  as a fixed world-space vector, so under a large rotation the detail is faithfully reproduced in entirely the wrong orientation.

The artifact is distinctive: bend a cylinder sharply and the surface features point the way they did before the bend, as though the detail had been painted on and the paint had not moved.

For small deformations this is invisible and the linear method is excellent. For large rotations it is unusable, which is what motivates §3.

### 3 As-rigid-as-possible deformation

Allow each vertex an unknown local rotation  $R_i$ , and ask that each one-ring neighbourhood deform as close to rigidly as the constraints permit.

$$E = \sum_i \sum_{j \in N(i)} w_{ij} \| (x'_i - x'_j) - R_i (x_i - x_j) \|^2$$

This is nonlinear — both the positions and the rotations are unknown — but it has a structure that makes it tractable: fixing either set makes solving for the other easy.

#### 3.1 The local step

With positions fixed, each  $R_i$  is independent and has a closed-form solution. Finding the rotation that best aligns one set of vectors with another is the **orthogonal Procrustes problem**, solved by an SVD of the covariance matrix.

```
Matrix3d S = Matrix3d::Zero();
for (int j : one_ring(i))
    S += w(i,j) * (x[i] - x[j]) * (xp[i] - xp[j]).transpose();

JacobiSVD<Matrix3d> svd(S, ComputeFullU | ComputeFullV);
Matrix3d R = svd.matrixV() * svd.matrixU().transpose();

if (R.determinant() < 0) {           // reflection, not rotation
    Matrix3d V = svd.matrixV();
    V.col(2) *= -1;                  // flip the least significant axis
    R = V * svd.matrixU().transpose();
}
```

#### THE DETERMINANT GUARD IS NOT OPTIONAL

$V U^T$  is orthogonal, which means it is either a rotation (determinant +1) or a **reflection** (determinant -1).

Under strong compression or a nearly degenerate configuration, the SVD can produce a reflection. Applying it turns that patch of surface inside out, producing locally inverted geometry that shades black and self-intersects.

Flipping the sign of the column corresponding to the smallest singular value yields the closest genuine rotation. Omitting this check is a classic bug, and the symptom — isolated inverted patches appearing under large deformation — is baffling without knowing the cause.

#### 3.2 The global step

With rotations fixed, the energy is quadratic in the positions and its minimiser solves a sparse linear system — the *same* Laplacian system as before, with rotated differential coordinates on the right-hand side.

Alternate the two steps. The energy decreases monotonically and the process typically converges in three to ten iterations. Crucially, the system matrix is unchanged throughout, so **one factorisation serves every iteration and every frame of dragging**. ARAP is therefore interactive despite being nonlinear, which is

the reason it became the standard method.

## 4 The wider toolkit

| Method                                                 | How it is controlled                                                                              | Suited to                                                                                                                       |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Laplacian / ARAP</b>                                | Handles placed directly on the surface.                                                           | Detail-preserving editing of scanned or sculpted models.                                                                        |
| <b>Cage-based</b> (mean value or harmonic coordinates) | A coarse polyhedron enclosing the model; moving cage vertices deforms everything inside smoothly. | Smooth large-scale deformation. Cheap to evaluate once the coordinates are precomputed.                                         |
| <b>Skeleton / linear blend skinning</b>                | A bone hierarchy with per-vertex weights.                                                         | Character animation. CSCE 641 Module 13.                                                                                        |
| <b>Free-form deformation</b>                           | A lattice of control points, with the model embedded in a trivariate Bézier or B-spline volume.   | Simple global warps: squash, stretch, bend. Historically the first practical method.                                            |
| <b>Blend shapes</b>                                    | Linear interpolation between explicitly sculpted target shapes.                                   | Facial animation. The result is exactly what an artist sculpted, which is why it remains dominant where appearance is critical. |

### THE IDEA THIS COURSE SHARES WITH CSCE 641

Linear blend skinning averages **transformation matrices**, and CSCE 641 Module 13 showed the consequence: the average of two rotation matrices  $180^\circ$  apart is degenerate, so a twisted wrist collapses — the candy-wrapper artifact.

The ARAP local step is the principled alternative stated plainly: do not average the matrices, *recover the best rotation* and apply that. The SVD is precisely the operation that extracts a rotation from something that is nearly one.

Dual-quaternion skinning is the real-time version of the same insight — interpolate the transformations rather than their matrix representations.

Two courses, two contexts, one fact: **rotations do not average linearly**, and every method that works respects that.

## 5 Where this leaves you

You can build a half-edge mesh and have it survive real, broken, scanned input. You can compute discrete curvature and verify it against a theorem rather than against intuition. You can build the cotangent Laplacian and use the same matrix for smoothing, for parameterisation, and for deformation — and you understand why one matrix serves all three. You can simplify, remesh, and unwrap an asset into something an engine will actually load.

That is a geometry processing library you wrote and understand completely, which is a far better foundation than familiarity with someone else's. The remaining track courses build directly on it: **CSCE 647** renders these surfaces with physically correct light transport; **CSCE 649** makes them move under simulated physics; **CSCE 748** reconstructs them from photographs.

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                                                           | What it covers for this module                                                    |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <a href="#">Sorkine &amp; Alexa — As-Rigid-As-Possible Surface Modeling (free)</a> | The ARAP paper. Short, clearly written, and directly implementable from the text. |
| <a href="#">Sorkine et al. — Laplacian Surface Editing (free)</a>                  | The linear method and the rotation problem, by the people who developed it.       |
| <a href="#">Keenan Crane — DDG, the deformation material</a>                       | Deformation framed in terms of the operators developed through the course.        |
| <a href="#">libigl tutorial — deformation chapter</a>                              | Working ARAP, biharmonic, and cage-based deformation to compare against your own. |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement naive editing with smooth falloff and capture the destruction of surface detail on a scanned model.
- 2 Implement linear Laplacian editing. Confirm that detail is preserved under translation of a handle.
- 3 Demonstrate the rotation problem: rotate a handle by 90° and capture the detail preserved in the wrong orientation.
- 4 Implement ARAP with the local SVD step and the global solve. Compare against the linear method under the same large rotation.
- 5 Remove the determinant guard from the SVD step and find a deformation that produces an inverted patch. Capture it.
- 6 Measure interactivity: time the factorisation and the per-drag solve separately on a 100,000-vertex mesh, and confirm dragging is interactive.
- 7 Implement cage-based deformation with mean value coordinates and compare the result and the cost against ARAP on the same model.

- 8 Project 2 is now due.** Complete the simplification, parameterisation, and remeshing pipeline described in the syllabus, with a baked normal map and everything rendered in your CSCE 641 renderer.

## Self-check

---

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does smoothly interpolating absolute positions destroy detail?
- 2 What are differential coordinates, and what earlier operator are they?
- 3 Why can the Laplacian editing factorisation be reused across handle moves?
- 4 Explain the rotation problem and describe its visual signature.
- 5 Describe the ARAP local and global steps, and say why the method is interactive despite being nonlinear.
- 6 Why must the determinant of the SVD result be checked, and what happens if you skip it?
- 7 What single fact about rotations connects this module to CSCE 641 Module 13?